



Topic 例程 使用说明

V1.5.2

2026.05

文档版本

版本号	修订日期
1.4	2025 年 07 月 15 日
1.5	2025 年 07 月 23 日
1.5.1	2026 年 01 月 09 日
1.5.2	2026 年 05 月 13 日

目录

文档版本	I
1.在 Linux 环境中使用 topic.....	3
2. topic 程序说明.....	5
附录	6

1.在 Linux 环境中使用 topic

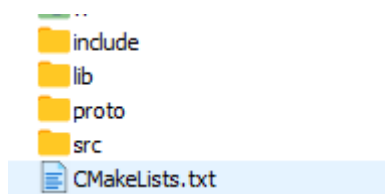
测试环境：

Cmake: 3.22.1
GCC: 9.4.0
Protoc: 3.21.12

1.1 解压 topic 例程压缩包

将压缩包拷贝到桌面并解压到 topic 文件夹中。

输入 ll 查看解压是否完成，若解压完成，则包含 CmakeList、include、lib、proto 和 src 文件及文件夹。



1.2 编译工程

创建 build 文件夹并进入

```
~/Desktop/topic_x86_c++$ mkdir build
~/Desktop/topic_x86_c++$ cd build
~/Desktop/topic_x86_c++/build$
```

使用 cmake .. 和 make -j4 指令编译工程。

```
~/Desktop/topic_x86_c++/build$ cmake ..
~/Desktop/topic_x86_c++/build$ make -j4
```

1.3 运行程序

输入：ll

并出现如下文件

```
Topic_SDK/topic_c++/build$ ll
4096 5月 15 19:53 ./
4096 5月 15 19:52 ./.
13901 5月 15 19:53 CMakeCache.txt
4096 5月 15 19:53 CMakeFiles/
1536 5月 15 19:53 cmake_install.cmake
4096 5月 15 19:53 generated_proto/
7765 5月 15 19:53 Makefile
893528 5月 15 19:53 topic*
Topic_SDK/topic_c++/build$
```

输入：./topic 运行程序，可返回状态信息及产品当前参数

```

NodeOptions
  name: {test1}
  pub url: {}
  sub url: {tcp://127.0.0.1:19091}
  thread poll size: {1}
  pub queue length: {50}
  sub queue length: {1000}
init
on_monitor_started
zmq_version 4.3.6
sub_monitor_thread_ started.
[SUB] connect: {tcp://127.0.0.1:19091}
Creating subscription for topic: system_rtstate
NodeOptions
  name: {test2}
  pub url: {}
  sub url: {tcp://127.0.0.1:19091}
  thread poll size: {1}
  pub queue length: {50}
  sub queue length: {1000}
LoopRecv started...
init
on_monitor_started
zmq_version 4.3.6
[SUB] connect: {tcp://127.0.0.1:19091}
Creating subscription for topic: system_nrtstate
sub_monitor_thread_ started.
LoopRecv started...

header:5761386 5747
system_running_state:1
controller:ethercat_controller 1      5761387 OK
slave 0      0 0 0 0      1      0
slave 1      1 0 0 0      1      0
slave 2      2 0 0 0      1      0
slave 3      3 0 0 0      1      0
slave 4      4 0 0 0      1      0
slave 5      5 0 0 0      1      0
model0 UR5e Model
current_point:

0 0 1 none none 0

interface0 0 1
interface1 1 1
interface2 2 1
interface3 3 1
interface4 4 1

```

若报如下错误，则需将\topic_demo\lib\linux\Release 中的 libzmq.so.5 文件复制到根目录的 lib 文件夹中

```

z@z-virtual-machine:~/桌面/test/message/build$ ./topic_sub
./topic_sub: error while loading shared libraries: libzmq.so.5: cannot open shared
object file: No such file or directory

```

若出现 libprotobuf.so.32 的报错，相同操作即可

注：此 demo 依赖于 protobuf 3.21.12，若报如图错误，说明没有安装此版本的 protobuf，安装后方可编译

```

-- Detecting CXX compile features
-- Detecting CXX compile features - done
CMake Error at CMakeLists.txt:25 (message):
  protoc not found

```

若运行后出现的“段错误”的问题，需要检查 GCC 版本。

```

pub queue length: {50}
sub queue length: {1000}

init
on_monitor_started
zmq_version 4.3.6
[SUB] connect: {tcp://192.168.11.11:19091}
Creating subscription for topic: system_nrtstate
sub_monitor_thread_ started.
LoopRecv started...
段错误 (核心已转储)

```

2. topic 程序说明

程序分为几个部分：配置 ip，订阅远程地址，实时数据处理获取，打印硬件模型资源等信息。

程序通过轮询全局状态接口获取数据。它的主循环中以高频率（500ms）调用 getRt () 接口实时获取 system_rtstate，用于监控机器人瞬时状态；同时在同一循环中调用 getNrt () 接口获取 system_nrtstate，用于读取系统配置与静态参数。RT 数据更新速度快，反映当前时刻的状态，而 NRT 数据更新相对较慢，主要用于系统配置管理。

程序中已经将所以接口信息全部打印，可以根据需求自行取用。

```
// 配置&创建节点 配置远程地址 192.168.2.10:19091 并创建topic节点
// 分别订阅 system_rtstate 及 system_nrtstate 主题
NodeOptions system_topic_options;
system_topic_options.node_name = "test1";
system_topic_options.sub_url = "tcp://192.168.11.11:19091";
Node system_topic_node(system_topic_options);
system_topic_node.Start();

===== RT =====
auto subrt = system_topic_node.CreateSubscription<system_rtstate::SystemRtState>("system_rtstate", [](const system_rtstate::SystemRtState& tt) [...]);

===== NRT =====
auto subnrt = system_topic_node.CreateSubscription<system_nrtstate::SystemNrtState>("system_nrtstate", [](const system_nrtstate::SystemNrtState& tt) [...]);

===== RT Basic =====
auto subrt_basic = system_topic_node.CreateSubscription<system_rtstate::SystemRtState>("system_rtstate", [](const system_rtstate::SystemRtState& tt) [...]);

while (true) {
    std::this_thread::sleep_for(std::chrono::microseconds(20));
}
return 0;
```

```
auto subrt = system_rtstate_node.CreateSubscription<system_rtstate::SystemRtState>("system_rtstate", [](const system_rtstate::SystemRtState& tt) {
    SystemStateData rtparm;
    display_rt(tt, rtparm);

    //=====
    std::cout << "1.rtheadr: " << rtparm.header_timestamp << " " << rtparm.header_frame_id << std::endl;
    std::cout << "2.system_running_state: " << rtparm.system_running_state << std::endl;

    std::cout << "3.Controller Info:" << std::endl;
    std::cout << " 3.1.controller_name: " << rtparm.controller.controller_name << std::endl;
    std::cout << " 3.2.control_cycle: " << rtparm.controller.control_cycle << std::endl;
    std::cout << " 3.3.global_count: " << rtparm.controller.global_count << std::endl;
    std::cout << " 3.4.master_info: " << rtparm.controller.master_info << std::endl;
    std::cout << " 3.5.is_link_up: " << rtparm.controller.is_link_up << std::endl;

    std::cout << "4.Models Info:" << std::endl;
    for (size_t i = 0; i < rtparm.models.size(); ++i) {
        std::cout << " 4. " << i << ". Model " << i << ":" << std::endl;
        std::cout << " 4a.model_name: " << rtparm.models[i].model_name << std::endl;
        std::cout << " 4b.model_type: " << rtparm.models[i].model_type << std::endl;

        std::cout << " 5.Joints Info:" << std::endl;
        for (size_t j = 0; j < rtparm.models_joints.size(); ++j) {
```

注：std::count 的打印方式没有线程保护，直接观察打印信息会存在顺序错乱问题属于正常现象，现有打印方式仅提供各接口以及调用方法，可根据需求进行选取调用。

附录

法律声明

1. 版权声明

© 2025 贞实科技版权所有。未经书面授权，禁止以任何形式复制、传播本文件内容。

2. 免责条款

使用者应自行承担因以下情况导致的损失风险：

- 未按指引操作造成的系统配置错误
- 擅自修改动态库文件引发的程序崩溃（如 message.dll 等库非授权替换）
- 开发环境版本不匹配导致的编译失败（如 CMake 2.8 与 VS2022 兼容问题）

3. 数据隐私条款（Data Privacy）

产品运行过程中产生的日志文件，仅包含设备状态信息，不会收集用户个人信息。